

Contador de Frequências Usando Estruturas de Dados Avançadas

Thalisson Gesian Perdigão Rocha

¹Universidade Federal do Ceará (UFC) – Quixadá – CE - Brasil

`thaper.rocha@alu.ufc.br`

Abstract. *This project implements a C++ application for counting word frequencies in text files, using generic data structures such as balanced trees and hash tables with different collision strategies. The goal is to compare the performance of these structures in terms of execution time, number of comparisons, and rotations (for the balanced trees) during the operations performed. The application also incorporates features such as Unicode handling, Collator-based sorting, and custom iterators. The results allowed for an analysis of the efficiency of each structure and reinforced the importance of choosing the appropriate data structure according to the specific use case.*

Resumo. *Este projeto implementa uma aplicação desenvolvida em C++ para contagem de frequência de palavras em arquivos de texto, utilizando estruturas genéricas como árvores balanceadas e tabelas hash com diferentes estratégias de colisão. O objetivo é comparar o desempenho dessas estruturas em termos de execução, quantidade de comparações e rotações realizadas (para as árvores balanceadas) durante as operações utilizadas. A aplicação também incorpora recursos como manipulação de Unicode, ordenação com Collator e iteradores personalizados. Os resultados obtidos permitiram analisar a eficiência de cada estrutura e reforçar a importância da escolha adequada da estrutura de dados conforme o cenário de uso.*

1. Introdução

Este projeto tem como objetivo desenvolver uma aplicação em C++ capaz de contar a frequência de palavras presentes em arquivos de texto, empregando estruturas de dados avançadas como árvores AVL, árvores Rubro-Negras e Tabelas Hash com diferentes estratégias de colisão.

Esse tipo de estudo é importante para compreendermos as vantagens e limitações de diferentes estruturas de dados em contextos práticos, contribuindo para escolhas mais eficientes no desenvolvimento de aplicações que lidam com grandes volumes de dados.

2. Métricas Utilizadas

Para avaliar o desempenho das estruturas de dados implementadas, foram utilizadas métricas específicas durante a construção da tabela de frequências. As métricas obrigatórias incluem o número de comparações de chaves realizadas durante a montagem da estrutura, bem como o tempo total de construção da tabela.

Além disso, como métrica adicional opcional, foi incluído o número de rotações realizadas nas árvores balanceadas (AVL e Rubro-Negra). Essa métrica permite observar o custo estrutural associado à manutenção do balanceamento das árvores ao longo da inserção das palavras, contribuindo para uma análise mais aprofundada da eficiência dessas abordagens.

3. Estruturas De Dados Implementadas

As estruturas de dados desenvolvidas neste projeto foram implementadas de forma genérica, utilizando templates da linguagem C++, permitindo o armazenamento de pares formados por uma chave e um valor associados. Essa abordagem promove reutilização e flexibilidade, sendo possível empregar essas estruturas em diferentes contextos além do escopo deste projeto.

Cada estrutura foi desenhada para manter eficiência nas operações fundamentais do dicionário, como inserção, busca e remoção. Além disso, foram adaptadas para registrar métricas de desempenho, como o número de comparações e, no caso das árvores balanceadas, o número de rotações executadas. A seguir, apresenta-se uma breve descrição das quatro estruturas utilizadas:

3.1. Árvore AVL

A árvore AVL é uma estrutura binária de busca que mantém seu balanceamento por meio da verificação do fator de equilíbrio de cada nó, garantindo que a diferença entre as alturas das subárvores esquerda e direita seja de no máximo 1. Para preservar essa propriedade, a árvore realiza rotações específicas sempre que uma operação de inserção ou remoção a desequilibra. Como resultado, a altura da árvore se mantém proporcional ao logaritmo do número de elementos armazenados, o que favorece a previsibilidade do desempenho.

3.2. Árvore Rubro-Negra

A árvore Rubro-Negra é uma variação de árvore binária de busca balanceada que utiliza um conjunto de propriedades baseadas em cores para garantir uma altura controlada. Diferente da AVL, seu balanceamento é mais flexível, permitindo maior tolerância a desbalanceamentos temporários. Essa característica faz com que, na prática, a Rubro-Negra tenda a exigir menos rotações do que a AVL, o que pode resultar em um desempenho superior em operações com inserções ou remoções frequentes.

3.3. Tabela Hash Com Encadeamento Exterior

Nesta estrutura, os elementos são distribuídos em uma tabela por meio de uma função de hash, e colisões são tratadas por listas encadeadas: cada posição da tabela armazena uma lista contendo todos os elementos que compartilham o mesmo índice. Essa abordagem é simples de implementar, flexível quanto ao número de elementos, e tolera bem colisões, especialmente se a função de dispersão for adequada. A estrutura cresce dinamicamente, mantendo um bom desempenho mesmo em casos com grande volume de dados.

3.4. Tabela Hash Com Endereçamento Aberto

A tabela com endereçamento aberto lida com colisões de forma diferente: em vez de usar listas, ela procura uma nova posição livre dentro da própria tabela. O método utilizado

nesta implementação é o hashing duplo, em que duas funções de hash são combinadas para definir os passos de sondagem em busca de um espaço disponível. Essa técnica reduz a chance de agrupamentos e melhora a distribuição dos dados, sendo eficiente em termos de uso de memória.

4. Implementação Do Dicionário

A implementação do projeto foi realizada por meio de uma classe genérica `dictionary<T>`, que permite armazenar palavras e suas respectivas frequências usando diferentes estruturas de dados como backend (Árvore AVL, Árvore Rubro-Negra, Chained Hash Table, Open Hash Table). A estrutura utilizada é indicada por meio do template `T`, o que permite testar o desempenho de diferentes abordagens sem alterar a lógica principal do programa.

4.1. Estrutura de Dados Utilizada

A estrutura de dados concreta é passada como template para a classe `dictionary`. Isso permite comparar o comportamento e a eficiência de diferentes estruturas (como árvores balanceadas ou tabelas de dispersão) no armazenamento e busca de palavras. A interface da estrutura deve oferecer os seguintes métodos:

- `insert(palavra, valor)`
- `remove(palavra)`
- `contains(palavra)`
- `search(palavra)`
- `update(palavra, novoValor)`
- `clear()`
- Iteração com `begin()` e `end()`
- `isEmpty()`
- `getSize()`
- `getComparisons()`
- `getRotations()` (para as árvores balanceadas)

4.2. Funcionamento do Dicionário

A classe `dictionary` encapsula toda a lógica necessária para conseguir realizar os seguintes métodos:

- `void insert(const icu::UnicodeString& palavra)`
Esse método realiza a inserção de uma palavra no dicionário:
Se a palavra já estiver presente, sua frequência é incrementada.
Se a palavra ainda não existir, ela é inserida com valor inicial igual a 1.
A operação usa `search()` para verificar a existência da palavra e trata a exceção `std::out_of_range` caso ela não seja encontrada.
- `void remove(const icu::UnicodeString& palavra)`
Remove completamente uma palavra do dicionário, se ela existir.
Essa operação é direta: chama o método `remove` da estrutura `T`.
- `void contains(const icu::UnicodeString& palavra) const`
Verifica se uma determinada palavra está presente no dicionário, utilizando o método `contains` da estrutura `T`.
Retorna `true` se a palavra foi encontrada e `false` caso contrário.

- `int search(const icu::UnicodeString& palavra) const`
Retorna a frequência associada a uma palavra.
Se a palavra estiver presente, retorna seu valor.
Caso contrário, lança uma exceção `std::out_of_range`.
 - `void update(const icu::UnicodeString& palavra, int up)`
Atualiza a frequência associada a uma palavra.
Se a palavra estiver presente, seu valor será substituído por `up`.
Caso a palavra não exista, a operação é ignorada.
 - `void clear()`
Remove todas as palavras armazenadas no dicionário e reinicia o estado da estrutura T.
 - `bool isEmpty() const`
Verifica se o dicionário está vazio.
Retorna `true` se não há nenhuma palavra encontrada.
 - `int getSize() const`
Retorna o número total de palavras distintas armazenadas no dicionário.
 - `int getComparisons() const`
Retorna o número de comparações realizadas durante as operações.
 - `icu::UnicodeString gerarTabela()`
Gera uma tabela formatada com duas colunas: a primeira contém a palavra (preenchida com espaços até 15 caracteres); e a segunda contém sua frequência (preenchida com até 10 caracteres).
A função percorre a estrutura T com um loop baseado em iteradores.
 - `void salvarConteudo(const std::string& arquivo, std::chrono::milliseconds duracao, const std::string& estrutura)`
Salva todas as informações relevantes do dicionário em um arquivo `.txt`:
 - Estrutura utilizada;
 - Tamanho do dicionário;
 - Tempo total de execução (em milissegundos);
 - Número de comparações realizadas;
 - Número de rotações (se a estrutura T possuir `getRotations()`);
 - Tabela formatada com palavras e frequências.
- Usa a função auxiliar `helpers::registrarArquivo` para escrever no arquivo de saída.

4.3. Formato de Entrada e Saída de Dados

- Entrada: uma série de palavras lidas de um arquivo de texto são inseridas uma a uma no dicionário.
- Saída: é gerado um arquivo `.txt` contendo todas as informações relevantes que foram informadas na parte da explicação do método `salvarConteudo`.

4.4. Decisões de Implementação

- Uso de `UnicodeString` (da biblioteca ICU) para lidar corretamente com acentuação, caracteres especiais e suporte a diferentes idiomas.

- Tratamento de exceções: ao tentar acessar uma palavra inexistente, o método `insert()` captura a exceção `std::out_of_range` para evitar falhas e inserir a palavra corretamente.
- Formatação de saída: a tabela foi alinhada com preenchimento manual de espaços em cada célula para manter a legibilidade.

5. Testes Executados e Seus Resultados

Diversos arquivos de texto foram utilizados para avaliar o desempenho das estruturas de dados desenvolvidas. Os seguintes dados foram coletados nos resultados desses testes: tamanho da tabela, tempo de execução, quantidade de comparações e quantidade de rotações (para as árvores balanceadas). Aqui estão os resultados dos principais testes:

5.1. A Riqueza das Nações

5.1.1. Árvore AVL

- **Tamanho da tabela:** 10038 palavras
- **Tempo de execução:** 239 milissegundos
- **Quantidade de comparações:** 6573595
- **Quantidade de rotações:** 7498

5.1.2. Chained Hash Table

- **Tamanho da tabela:** 10038 palavras
- **Tempo de execução:** 103 milissegundos
- **Quantidade de comparações:** 475460

5.1.3. Open Hash Table

- **Tamanho da tabela:** 10038 palavras
- **Tempo de execução:** 102 milissegundos
- **Quantidade de comparações:** 513451

5.1.4. Árvore Rubro-Negra

- **Tamanho da tabela:** 10038 palavras
- **Tempo de execução:** 303 milissegundos
- **Quantidade de comparações:** 11264093
- **Quantidade de rotações:** 6318

5.2. Dom Casmurro

5.2.1. Árvore AVL

- **Tamanho da tabela:** 10069 palavras
- **Tempo de execução:** 64 milissegundos
- **Quantidade de comparações:** 1766156
- **Quantidade de rotações:** 7344

5.2.2. Chained Hash Table

- **Tamanho da tabela:** 10069 palavras
- **Tempo de execução:** 34 milissegundos
- **Quantidade de comparações:** 82445

5.2.3. Open Hash Table

- **Tamanho da tabela:** 10069 palavras
- **Tempo de execução:** 32 milissegundos
- **Quantidade de comparações:** 102323

5.2.4. Árvore Rubro-Negra

- **Tamanho da tabela:** 10069 palavras
- **Tempo de execução:** 75 milissegundos
- **Quantidade de comparações:** 2340004
- **Quantidade de rotações:** 6191

5.3. KJV Bible

5.3.1. Árvore AVL

- **Tamanho da tabela:** 12578 palavras
- **Tempo de execução:** 467 milissegundos
- **Quantidade de comparações:** 13340032
- **Quantidade de rotações:** 9517

5.3.2. Chained Hash Table

- **Tamanho da tabela:** 12578 palavras
- **Tempo de execução:** 185 milissegundos
- **Quantidade de comparações:** 1023535

5.3.3. Open Hash Table

- **Tamanho da tabela:** 12578 palavras
- **Tempo de execução:** 182 milissegundos
- **Quantidade de comparações:** 1130858

5.3.4. Árvore Rubro-Negra

- **Tamanho da tabela:** 12578 palavras
- **Tempo de execução:** 600 milissegundos
- **Quantidade de comparações:** 23167348
- **Quantidade de rotações:** 8010

5.4. O Manifesto Comunista

5.4.1. Árvore AVL

- **Tamanho da tabela:** 2628 palavras
- **Tempo de execução:** 14 milissegundos
- **Quantidade de comparações:** 343385
- **Quantidade de rotações:** 1918

5.4.2. Chained Hash Table

- **Tamanho da tabela:** 2628 palavras
- **Tempo de execução:** 9 milissegundos
- **Quantidade de comparações:** 18304

5.4.3. Open Hash Table

- **Tamanho da tabela:** 2628 palavras
- **Tempo de execução:** 9 milissegundos
- **Quantidade de comparações:** 22800

5.4.4. Árvore Rubro-Negra

- **Tamanho da tabela:** 2628 palavras
- **Tempo de execução:** 16 milissegundos
- **Quantidade de comparações:** 414415
- **Quantidade de rotações:** 1577

5.5. Sherlock Holmes

5.5.1. Árvore AVL

- **Tamanho da tabela:** 8357 palavras
- **Tempo de execução:** 77 milissegundos
- **Quantidade de comparações:** 2192517
- **Quantidade de rotações:** 6049

5.5.2. Chained Hash Table

- **Tamanho da tabela:** 8357 palavras
- **Tempo de execução:** 38 milissegundos
- **Quantidade de comparações:** 128680

5.5.3. Open Hash Table

- **Tamanho da tabela:** 8357 palavras
- **Tempo de execução:** 37 milissegundos
- **Quantidade de comparações:** 151589

5.5.4. Árvore Rubro-Negra

- **Tamanho da tabela:** 8357 palavras
- **Tempo de execução:** 94 milissegundos
- **Quantidade de comparações:** 3255004
- **Quantidade de rotações:** 5054

5.6. O Jardim Secreto

5.6.1. Árvore AVL

- **Tamanho da tabela:** 5315 palavras
- **Tempo de execução:** 56 milissegundos
- **Quantidade de comparações:** 1599085
- **Quantidade de rotações:** 3992

5.6.2. Chained Hash Table

- **Tamanho da tabela:** 5315 palavras
- **Tempo de execução:** 27 milissegundos
- **Quantidade de comparações:** 105718

5.6.3. Open Hash Table

- **Tamanho da tabela:** 5315 palavras
- **Tempo de execução:** 25 milissegundos
- **Quantidade de comparações:** 118825

5.6.4. Árvore Rubro-Negra

- **Tamanho da tabela:** 5315 palavras
- **Tempo de execução:** 68 milissegundos
- **Quantidade de comparações:** 2401946
- **Quantidade de rotações:** 3235

6. Como Executar este Programa

Para conseguir compilar e executar este projeto corretamente, siga as instruções abaixo:

6.1. Instalar Bibliotecas ICU

Para conseguir executar este projeto, é importante e necessário instalar as bibliotecas ICU no seu dispositivo. No caso do Ubuntu, você consegue instalar as bibliotecas ICU utilizando o comando abaixo:

```
sudo apt-get install libicu-dev
```

6.2. Como Compilar este Projeto

Utilize o seguinte comando para compilar este projeto corretamente:

```
cd ProjetoFinal
```

```
g++ -std=c++17 principais/main.cpp principais/helpers.cpp  
-o main -licuuc -licui18n
```

6.3. Como Iniciar o Executável

Para conseguir utilizar o executável **main**, é necessário passar dois argumentos pela linha de comando: a estrutura utilizada e o nome do arquivo de texto. A seguir, mostro um exemplo de como executar este programa:

```
./main 'estrutura' 'arquivo'
```

Exemplo: `./main avl exemplo.txt`

Vale ressaltar que as estruturas disponíveis são essas aqui:

- **Árvore AVL:** avl
- **Chained Hash Table:** chained
- **Open Hash Table:** open
- **Árvore Rubro-Negra:** rb

6.4. Instruções de Uso e como Acessá-las

O programa aceita um argumento especial para exibir instruções de uso e as opções disponíveis para as estruturas de dados. Para visualizar as instruções, execute o programa com o argumento `--help`, da seguinte forma:

```
./main --help
```

Ao executar este comando, o programa exibirá no terminal informações como:

```
Instruções: ./main 'estrutura' 'arquivo'
```

```
Estruturas disponíveis:
```

```
avl      -> Árvore AVL  
rb       -> Árvore Rubro-Negra  
chained  -> Tabela Hash Com Encadeamento  
open     -> Tabela Hash Com Endereçamento Aberto
```

Exemplo de uso: `./main avl exemplo.txt`

Além disso, o programa verifica se os argumentos fornecidos são corretos. Caso o número de argumentos esteja incorreto, uma mensagem de erro será exibida, sugerindo o uso do argumento `--help` para consultar as instruções.

6.5. Como Organizar os Arquivos Corretamente

Os arquivos de entrada precisam estar na pasta `testesFinais/entrada` para que sejam acessados pelo programa de forma correta.

Ao final da execução do programa, o novo arquivo gerado será inserido na pasta `testesFinais/saida`.

7. Dificuldades Encontradas

Durante a implementação deste projeto, alguns pontos me exigiram um estudo mais aprofundado e cuidadoso. As principais dificuldades que enfrentei foram:

- Verificação condicional com `type_traits`: a inclusão da métrica de rotações só faz sentido para as estruturas de árvores balanceadas (AVL e Rubro-Negra). Então, para evitar qualquer erro de compilação com as estruturas que não possuem o `getRotations()` (Tabelas Hash), eu tive que utilizar `type_traits` com `std::void_t` e SFINAE (Substitution Failure Is Not An Error). Com isso, foi possível criar a `trait has_getRotations;T;` e condicionar trechos do código usando `if constexpr`, para garantir segurança e manter o código genérico.
- Manipulação de `UnicodeString` e ordenação com o Collator: como o projeto precisava lidar com palavras em português e acentuação, foi necessário utilizar a classe `UnicodeString` para garantir que essas questões fossem tratadas corretamente. Essa biblioteca ICU foi muito importante para o funcionamento deste projeto, porém, ela me exigiu muito cuidado, principalmente na parte da conversão de `UnicodeString` para `std::string`. Além disso, para que fosse garantida a ordenação correta das palavras, foi necessário criar um comparador personalizado (`comparing`) usando o Collator. Tudo isso exigiu entendimento de como a biblioteca realiza essas comparações e como integrá-las em estruturas genéricas com o operador de comparação.
- Criação dos iteradores: Para que o dicionário fosse compatível com o range-based `for`, eu precisei criar iteradores específicos para cada estrutura. Isso exigiu que eu implementasse corretamente os operadores como: `operator!=` (para verificar desigualdade), `operator*` (para desreferenciar e acessar o par), `operator->` (para acessar via ponteiro), `operator++` (para avançar o iterador). No caso das árvores, o iterador precisava ser percorrido manualmente pelo percurso `in-order`, enquanto na tabela hash foi necessário controlar os índices e acesso ao vetor de chaves ordenadas (`getOrdKeys()`) e às marcações de atualização (`dirtyKeys`).
- Sobrecarga do operador `()`: em algumas partes do projeto, principalmente na criação de funções de hash e do comparador personalizado, foi necessário sobrecarregar o operador `()` em algumas structs (`hashUni` e `comparing`). Por exemplo, a `hashUni` que permitiu que a `UnicodeString` fosse utilizada como chave nas funções de hashing, pois o `std::hash` não possui uma sobrecarga padrão para o `UnicodeString`, então foi necessário que eu implementasse manualmente esse operador usando o `hashCode()` fornecido pela ICU. Essa técnica, por mais que eu tenha percebido que é algo comum em projetos mais avançados, me exigiu uma compreensão mais profunda da sintaxe e de como esses objetos functor funcionam.

- Implementação da Open Hash Table: a tabela hash por endereçamento aberto foi uma parte que me deu bastante trabalho, principalmente pela parte do cálculo do índice com hashing duplo, que foi necessário implementar uma segunda função de hash para resolver as colisões, o que me exigiu muito cuidado. Outras partes, como o redimensionamento (rehash), a manutenção do loadFactor e maxLoadFactor, e a ordenação das chaves para a iteração, foram algumas das partes da implementação da Open Hash Table que também me exigiram um pouco mais de cuidado, porém, foram menos complexos que a parte do hashing duplo.

8. Conclusão

A realização deste projeto proporcionou uma oportunidade valiosa para aprofundar o entendimento sobre diferentes estruturas de dados e suas aplicações práticas no contexto de contagem de palavras e suas frequências. Mais do que apenas implementar estruturas como árvores balanceadas e tabelas hash, foi possível observar na prática como características internas, como balanceamento, tratamento de colisões e organização da memória, afetam diretamente o desempenho de um sistema.

Além disso, este projeto foi uma experiência bastante completa para o aprendizado, indo muito além da implementação das estruturas genéricas. Ao longo do desenvolvimento deste projeto, foram explorados alguns conceitos mais avançados do C++, como o uso da biblioteca ICU para manipular os textos corretamente por meio da `UnicodeString`, além da criação de functors personalizados para hashing e ordenação, e aplicação de `type_traits` com `std::void_t` para adaptar algumas funcionalidades. Todas essas implementações contribuíram para tornar este projeto mais robusto e completo.

Durante os testes, ficou claro que as árvores balanceadas tendem a realizar um número significativamente maior de comparações, principalmente devido às exigências de balanceamento mais rígidas. Esse custo adicional se reflete no tempo gasto para realizar operações como inserção e busca, uma vez que o algoritmo precisa garantir que a estrutura permaneça balanceada após cada modificação. Como consequência, esses fatores acabam afetando o desempenho dessas estruturas em situações com grandes volumes de dados ou operações frequentes.

As tabelas hash se destacaram como as estruturas mais eficientes no geral, tanto em tempo de execução quanto em número de comparações. A diferença entre a Open Hash Table e a Chained Hash Table foi pequena, variando mais conforme o ambiente de execução. Ainda assim, a tabela com endereçamento aberto mostrou-se uma opção mais enxuta em termos de memória, por não depender de ponteiros extras. Apesar de exigir uma ordenação à parte para a iteração, o impacto dessa ordenação foi baixo mesmo em testes maiores, o que reforça sua eficiência.

Ao final, fica evidente que a melhor estrutura a ser utilizada depende diretamente do contexto do problema. Se o foco está em inserção e busca rápidas, as tabelas hash são as mais indicadas, já que conseguem acessar diretamente a posição de uma palavra com poucas comparações, mesmo em grandes volumes de dados. Por outro lado, se for necessário manter as palavras organizadas, por exemplo, para exibir uma lista ordenada em ordem alfabética ou percorrer os dados nessa sequência, as árvores se destacam. Mais do que apenas comparar números, este projeto reforçou a importância de entender bem as características de cada estrutura e saber escolher a ferramenta certa para cada situação.

9. Bibliografia

References

- [1] cppreference.com. (2025). *std::void_t*. Disponível em:
[cppreference.com/std::void_t](https://en.cppreference.com/std/void_t)
- [2] unicode-org.github.io. (2025). *icu::Collator Class Reference*. Disponível em:
[icu::Collator Class - unicode-org.github.io](https://unicode-org.github.io/icu::Collator%20Class)
- [3] daveparillo.github.io. (2025). *Binary Search Tree iterators — CISC 187 Course Reader Overview*. Disponível em:
[BST Iterators - daveparillo.github.io](https://daveparillo.github.io/BST%20Iterators)
- [4] geeksforgeeks.org. (2025). *Implementing Forward Iterator in BST*. Disponível em:
[Forward Iterator in BST - GeeksforGeeks](https://www.geeksforgeeks.org/implementing-forward-iterator-in-bst/)
- [5] unicode-org.github.io. (2025). *icu::UnicodeString Class Reference*. Disponível em:
[icu::UnicodeString Class - unicode-org.github.io](https://unicode-org.github.io/icu::UnicodeString%20Class)
- [6] medium.com. (2025). *Building your own HashMap in C++ — Open Addressing & Separate Chaining Implementations*. Disponível em:
<https://medium.com/...hashmap-in-c...>
- [7] cplusplus.com. (2017). *Custom iterator for a hash table — C++ Forum*. Disponível em:
[Custom iterator for a hash table - cplusplus.com](https://en.cppreference.com/iterator/custom-iterator-for-a-hash-table)
- [8] programiz.com. (2025). *C++ Functors*. Disponível em:
[C++ Functors - programiz.com](https://www.programiz.com/cplusplus/functors)